TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DEL VALLE DE OAXACA

INSTITUTO TECNOLOGICO DEL VALLE DE OAXACA

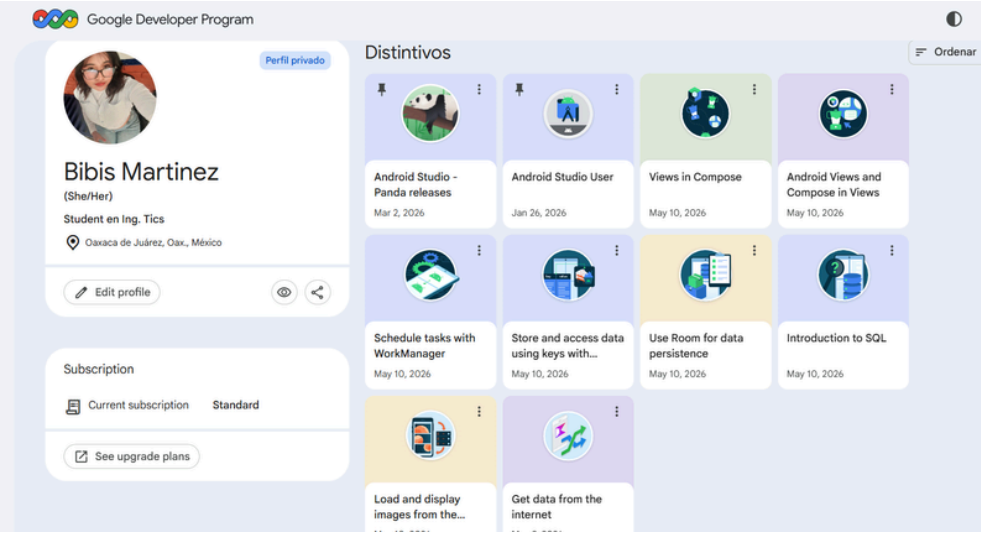
TECNOLOGIAS DE LA INFORMACION Y LAS COMUNICACIONES

DESARROLLO DE APLICACIONES MOVILES

4_1 BITÁCORA SOBRE MANIPULACIÓN DE DATOS (CRUD) (100%)

ALUMNA: ALEYDA BIBIANA MARTINEZ SANCHEZ

PROFESOR: AMBROSIO CARDOSO JIMENEZ



link https://github.com/bibis03/CRUD-Estadios-Bitacora

UNIDAD 4

TICS

SEMESTRE 6TO

29/05/2026

4_1 BITÁCORA SOBRE MANIPULACIÓN DE DATOS (CRUD)

CRUD DE ESTADIOS EN ROOM

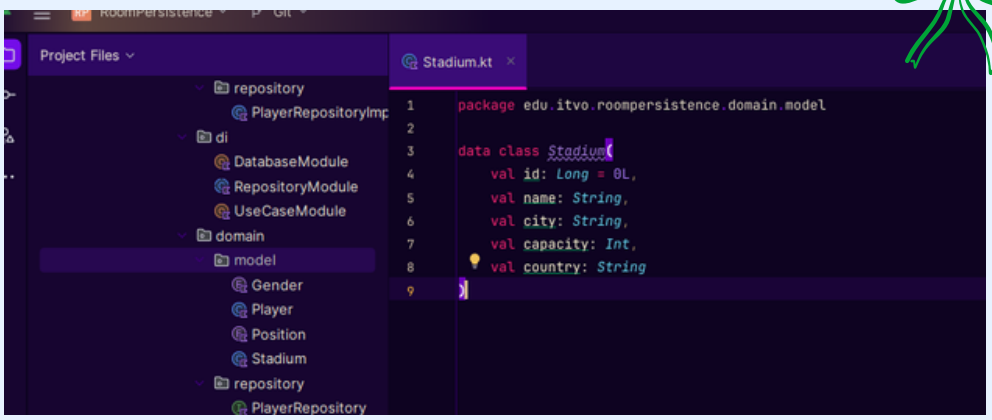
OBJETIVO

Agregar una nueva entidad llamada Stadium a la base de datos Room, manteniendo la misma arquitectura utilizada para Player.

PASO 1. CREAR STADIUM.KT

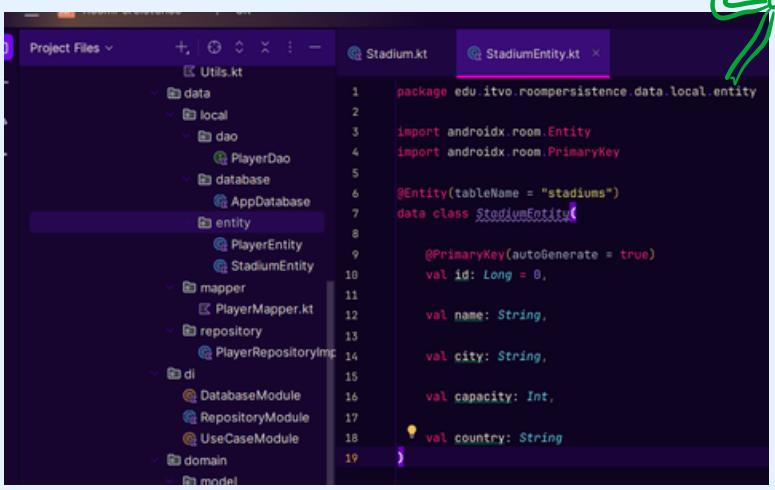
Se creó la clase Stadium dentro de la capa Domain para representar la información de los estadios.
Resultado
La clase almacena:

- id
- nombre
- ciudad
- capacidad
- país



PASO 2. CREAR STADIUMENTITY.KT

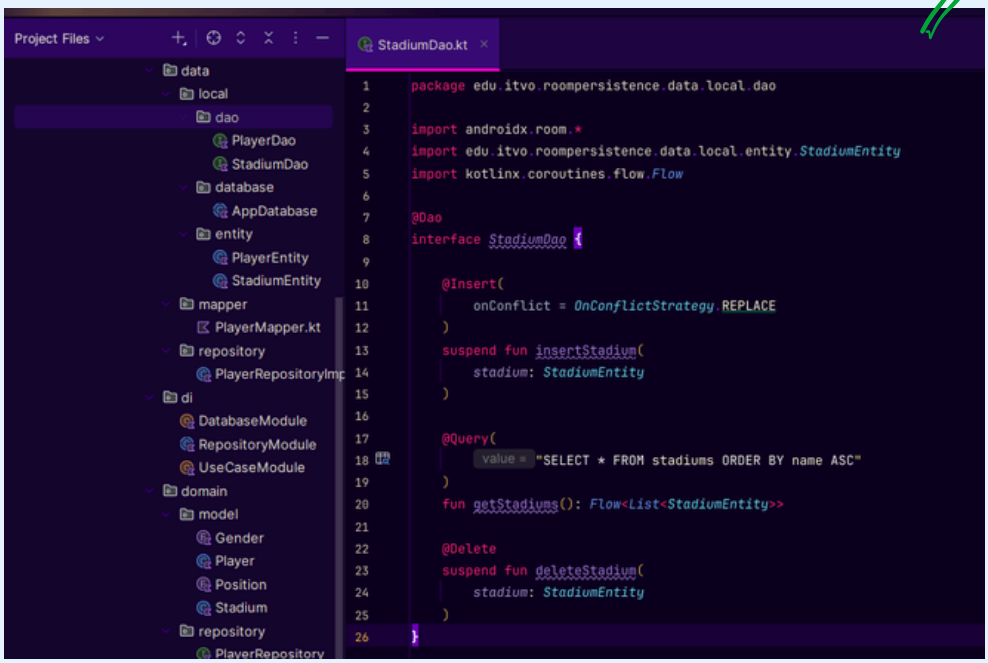
Se implementó una entidad Room para almacenar la información de los estadios dentro de la base de datos local.
Se creó la tabla:
stadiums



PASO 3. CREAR STADIUMDAO.KT

Se desarrolló el DAO para realizar operaciones CRUD sobre la tabla Stadiums.
Operaciones

- Insertar estadio
- Consultar estadios
- Eliminar estadio

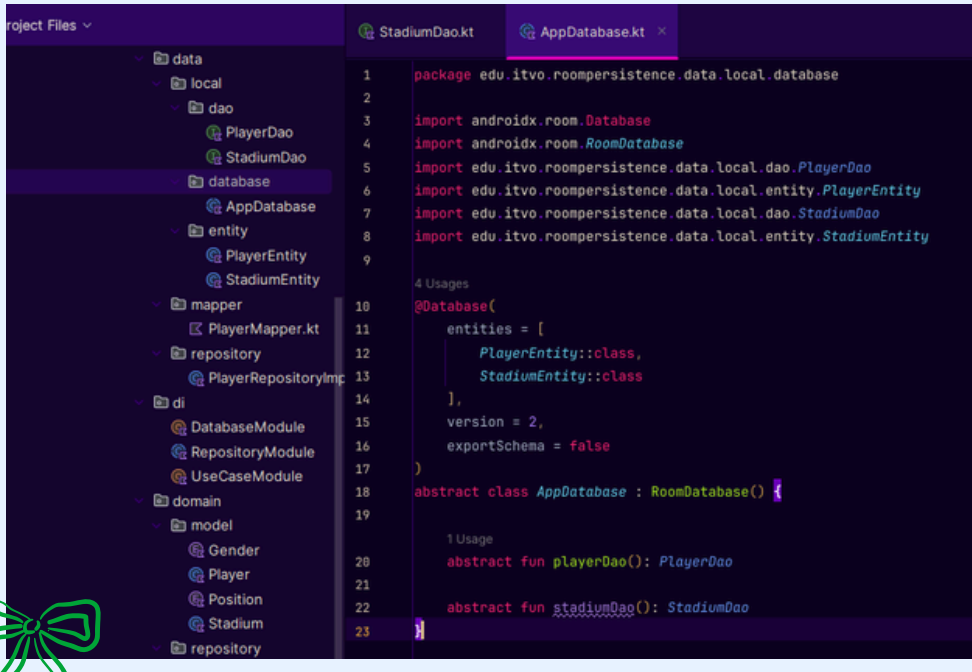


PASO 4. MODIFICAR APPDATABASE.KT

Se incorporó la entidad StadiumEntity al esquema de Room y se añadió el acceso mediante StadiumDao.
Resultado
La base de datos ahora administra:

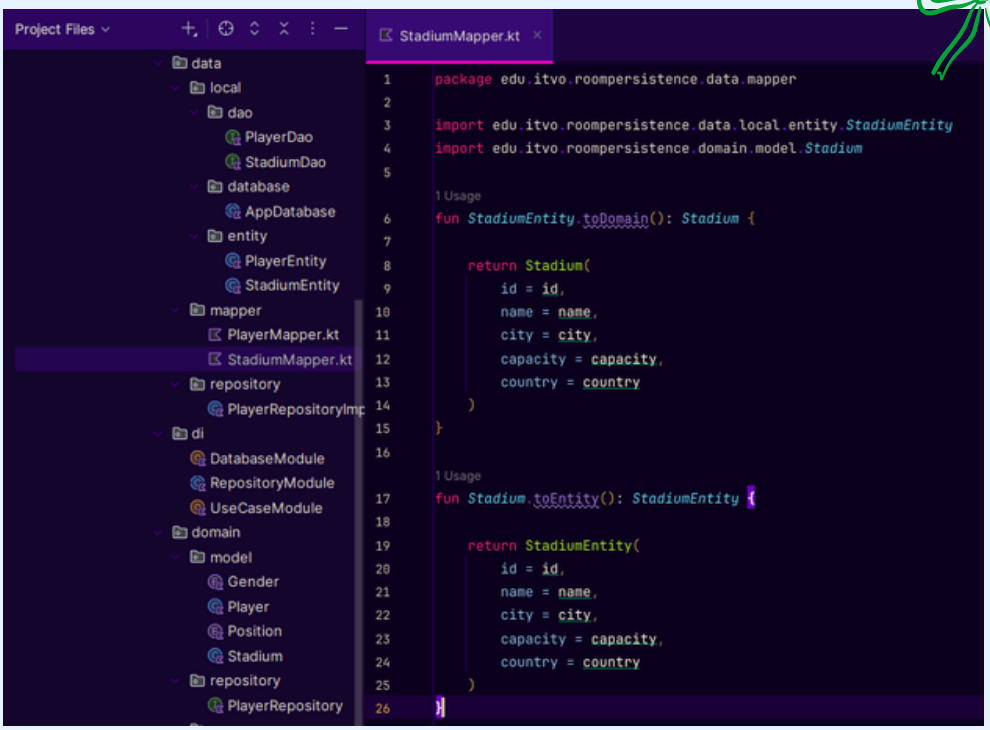
- Players
- Stadiums

Antes
entities = [PlayerEntity::class]



PASO 5. CREAR STADIUMMAPPER.KT

Se creó la conversión entre la entidad Room y el modelo de dominio para mantener separadas las capas de la aplicación.

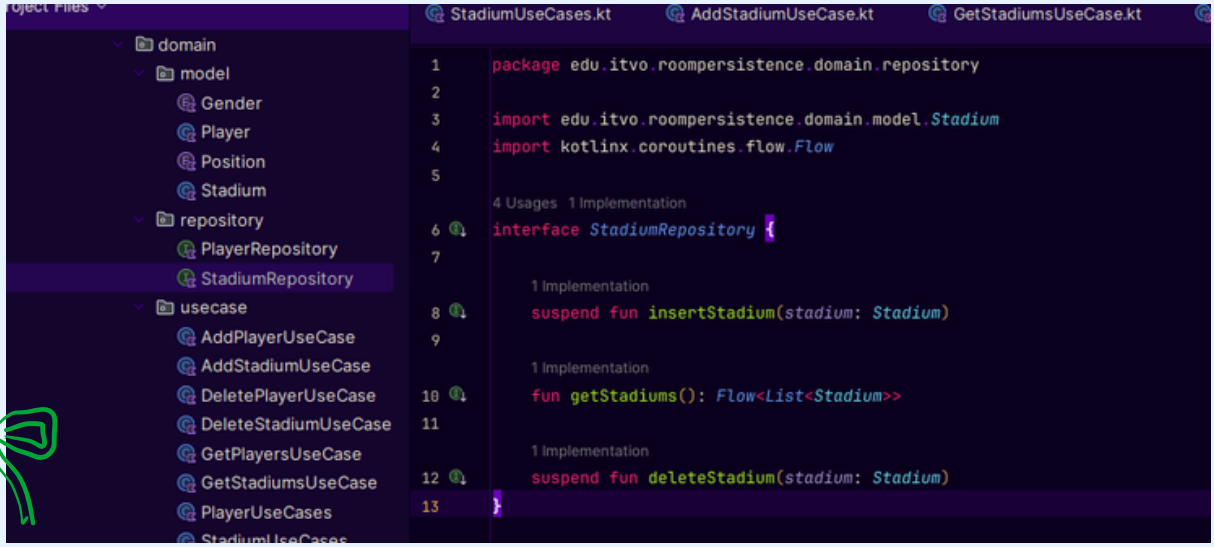


5.1 INTERFAZ STADIUMREPOSITORY

Se implementó la capa de Repository para manejar la comunicación entre la base de datos Room y la lógica de la aplicación.

Se creó una interfaz que define las operaciones básicas del CRUD.
Funciones:

- Insertar estadio
- Obtener lista de estadios
- Eliminar estadio

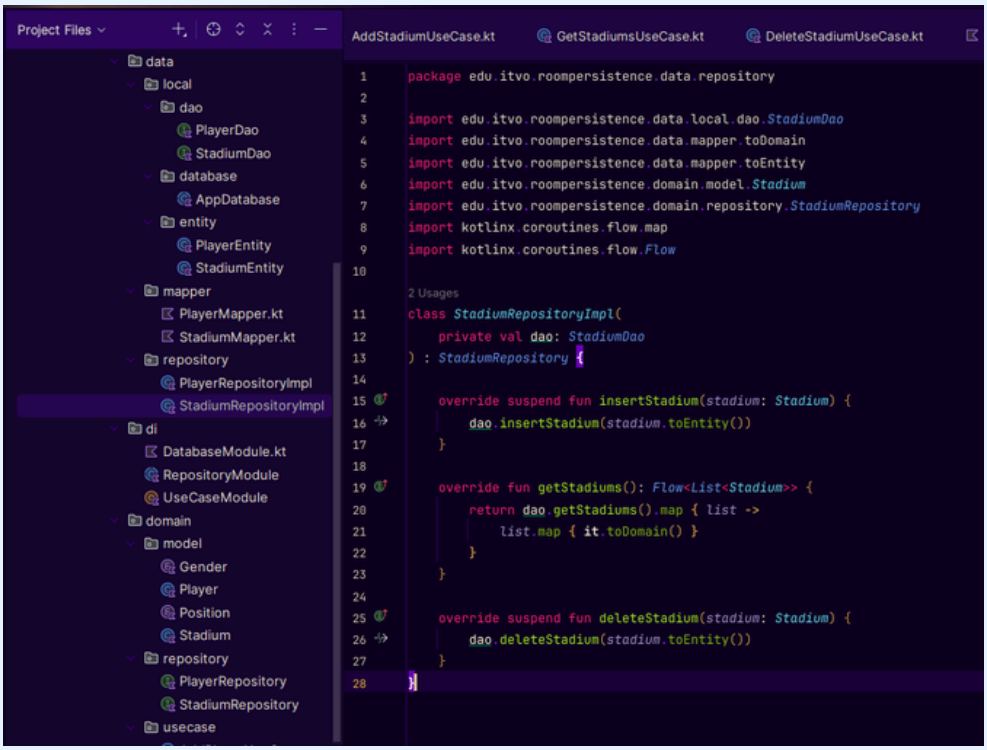


6 IMPLEMENTACIÓN STADIUMREPOSITORYIMPL

Se creó la implementación que conecta la interfaz con Room mediante el DAO.

Función:

- Convierte datos de dominio ↔ entidad
- Ejecuta consultas a la base de datos



RESULTADO

✔️ Se conectó la lógica de la app con la base de datos

✔️ Se implementó el patrón Repository

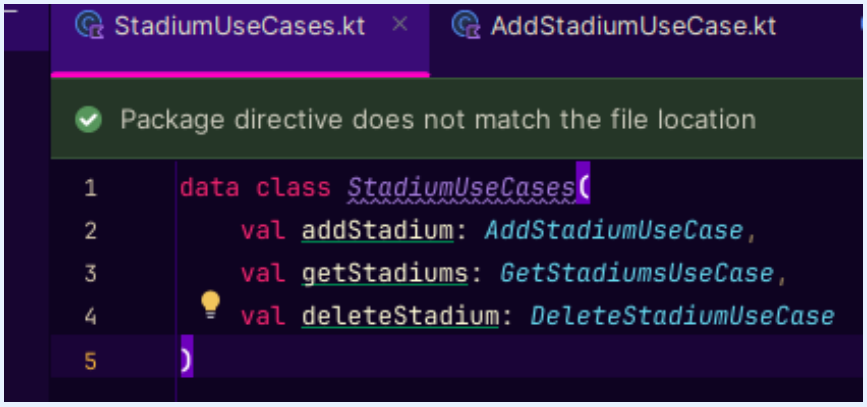
✔️ Se permitió el manejo de datos sin depender directamente de Room en la UI

7. CASOS DE USO (USE CASES)

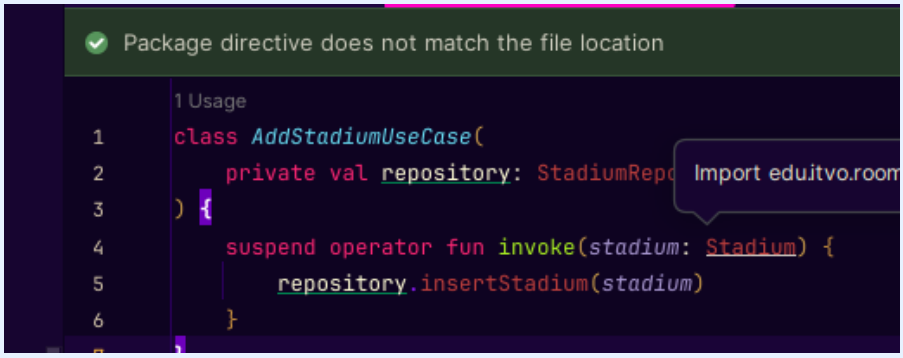
Se crearon los UseCases, que representan las acciones del sistema de forma independiente.
Esto mejora la organización del proyecto (Clean Architecture).

7.1 STADIUMUSECASES

Agrupa todas las operaciones del CRUD en un solo objeto.



7.2 ADDSTADIUMUSECASE



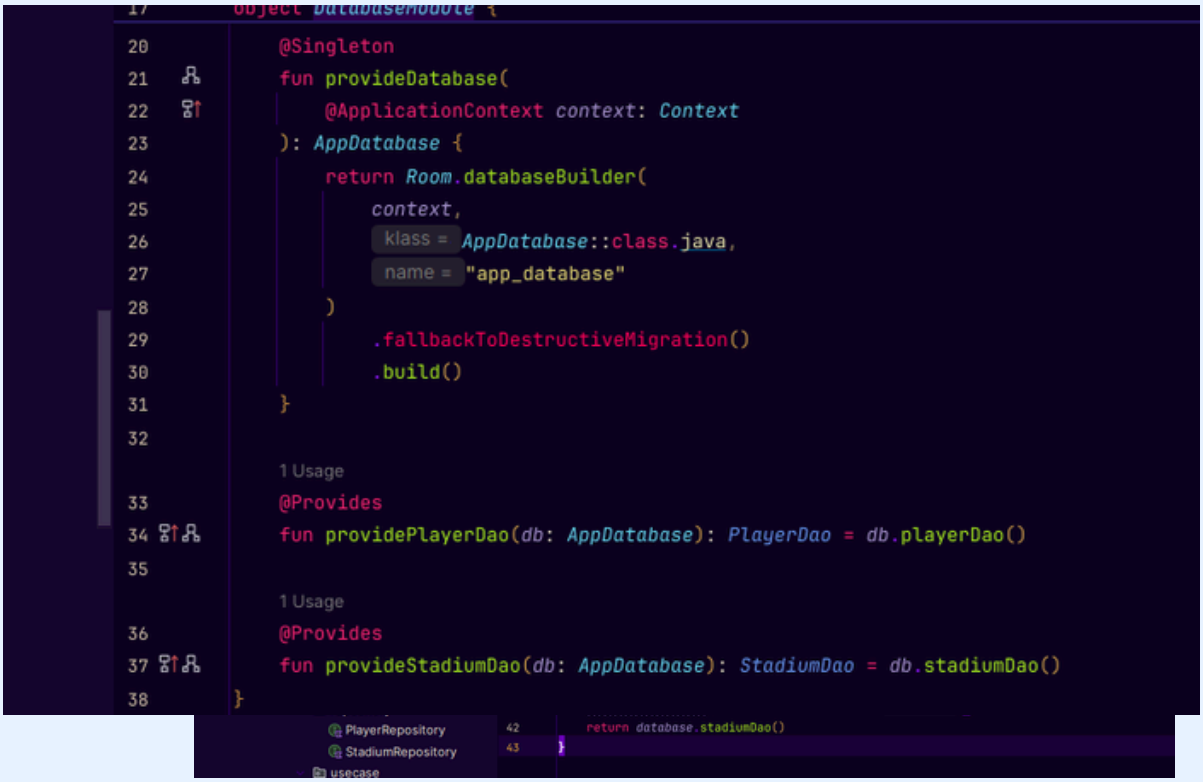
7.3 GETSTADIUMSUSECASE

8. INYECCIÓN DE DEPENDENCIAS (HILT)

Se configuró Hilt para que las clases se conecten automáticamente sin crear instancias manualmente

8.1 DATABASEMODULE (DAO)

Se agregó el DAO de Stadium a la base de datos.



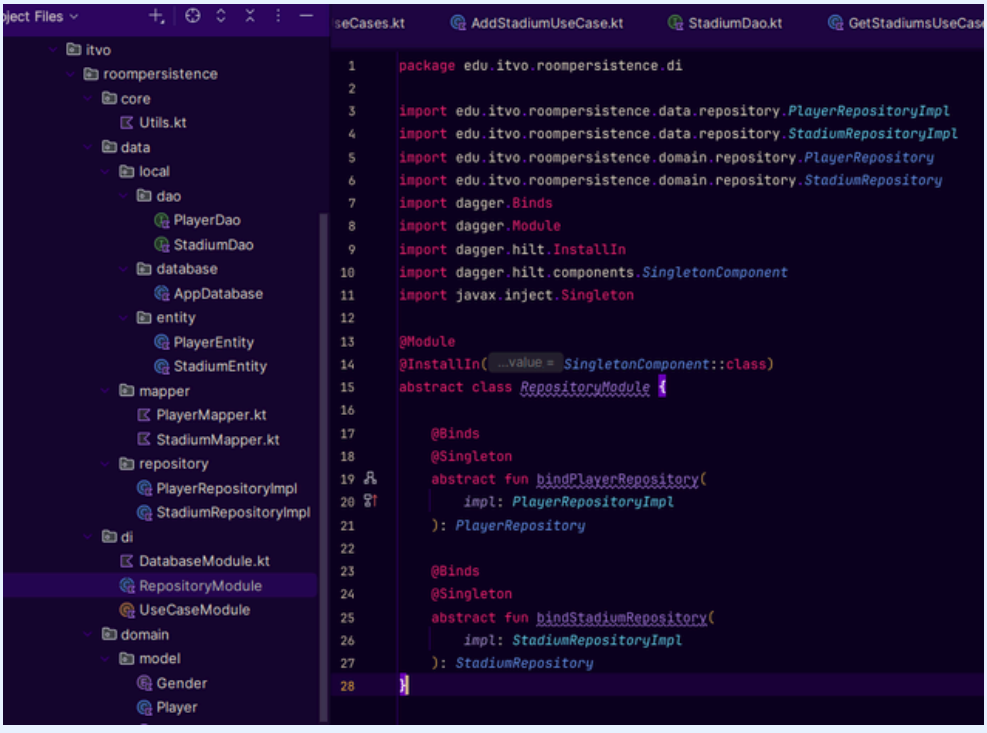
```
@Singleton
fun provideDatabase(
    @ApplicationContext context: Context
): AppDatabase {
    return Room.databaseBuilder(
        context,
        klass = AppDatabase::class.java,
        name = "app_database"
    )
        .fallbackToDestructiveMigration()
        .build()
}

1 Usage
@Provides
fun providePlayerDao(db: AppDatabase): PlayerDao = db.playerDao()

1 Usage
@Provides
fun provideStadiumDao(db: AppDatabase): StadiumDao = db.stadiumDao()
}
```

8.2 REPOSITORYMODULE

Se vinculó la interfaz con su implementación.



```
package edu.itvo.roompersistence.di

import edu.itvo.roompersistence.data.repository.PlayerRepositoryImpl
import edu.itvo.roompersistence.data.repository.StadiumRepositoryImpl
import edu.itvo.roompersistence.domain.repository.PlayerRepository
import edu.itvo.roompersistence.domain.repository.StadiumRepository
import dagger.Binds
import dagger.Module
import dagger.hilt.InstallIn
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton

@Module
@InstallIn( value = SingletonComponent::class)
abstract class RepositoryModule {

    @Binds
    @Singleton
    abstract fun bindPlayerRepository(
        impl: PlayerRepositoryImpl
    ): PlayerRepository

    @Binds
    @Singleton
    abstract fun bindStadiumRepository(
        impl: StadiumRepositoryImpl
    ): StadiumRepository
}
```

BOTÓN (AGREGAR ESTADIO)

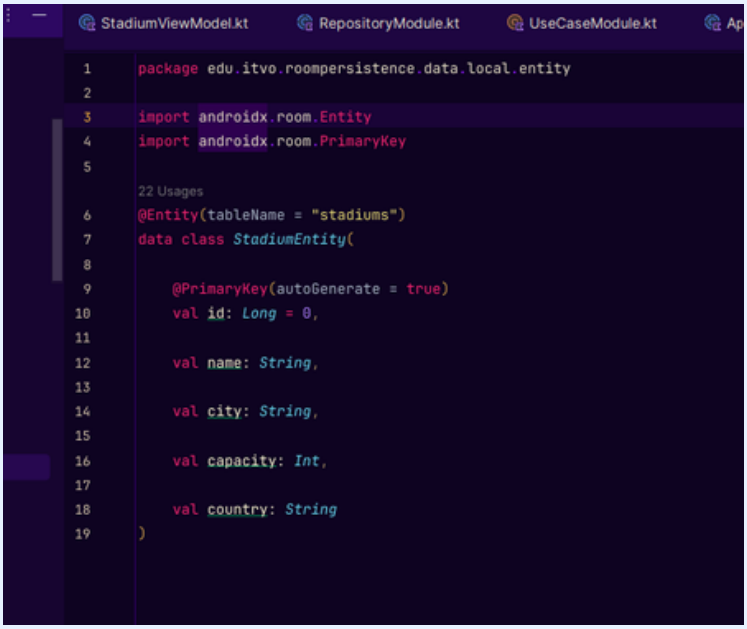
En tu StadiumScreen
Cuando el usuario presiona el botón:

- Se leen los textos de los TextFieldname
- city
- country
- capacity

1. Se crea un objeto:

Stadium(...)

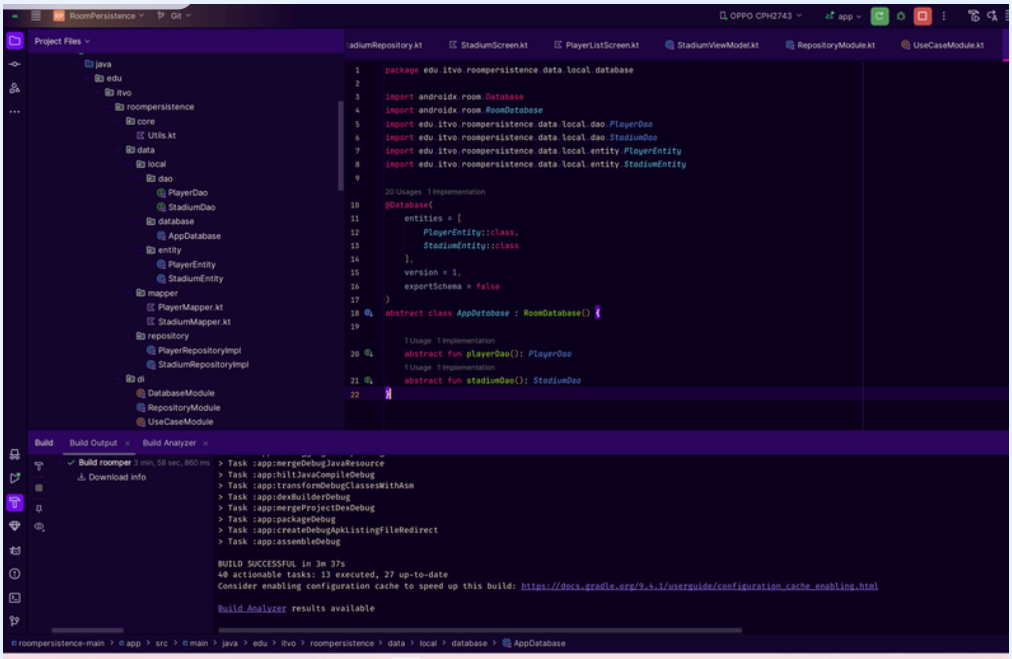
TABLA / COLUMNA EN BASE DE DATOS



BASE DE DATOS (ROOM)

AppDatabase.kt
Room toma el objeto
lo convierte en SQL
lo inserta en SQLite
Ejemplo interno:

INSERT INTO stadiums (name, city, capacity, country)
VALUES (...)



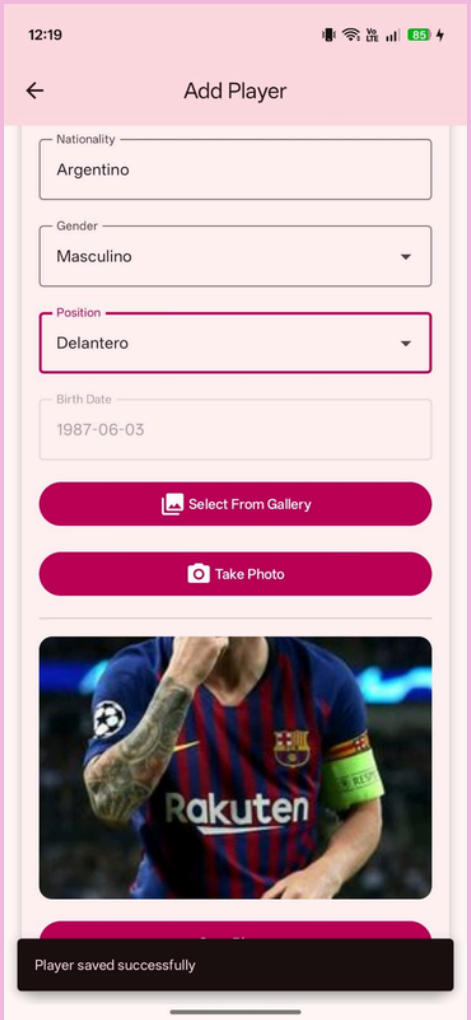
RESULTADO FINAL

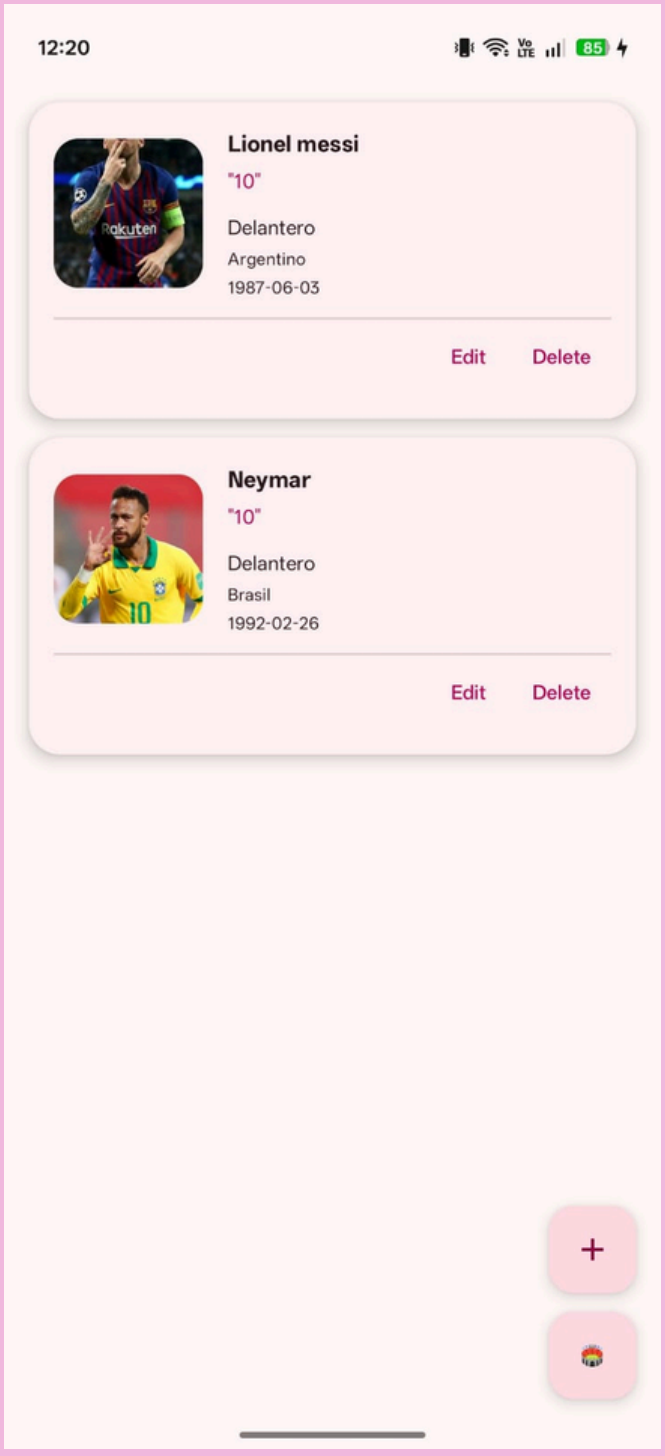
Ejecución del sistema CRUD con dos entidades: Players y Stadiums.
Descripción
Se ejecutó la aplicación Android con Room Persistence, integrando dos módulos:

- Gestión de jugadores
- Gestión de estadios

Se validó que ambas entidades funcionan de manera independiente dentro de la misma base de datos local.
Resultado
La aplicación permite:

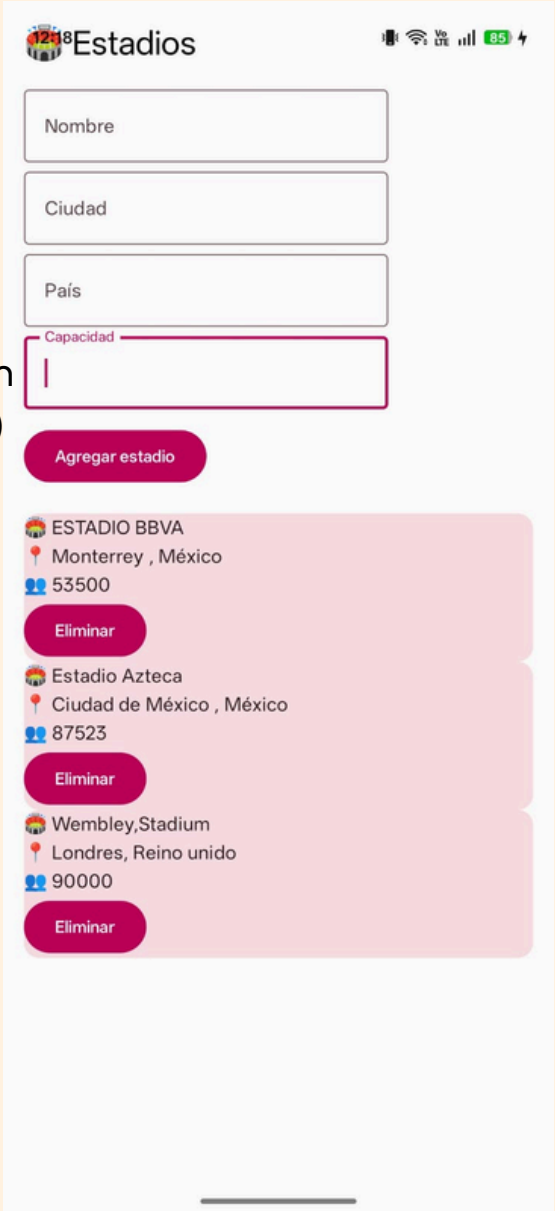
- Registrar jugadores
- Registrar estadios
- Visualizar listas dinámicas
- Eliminar registros





RESULTADO FINAL

¿Qué se logró?
✔️ Crear estadios nuevos
✔️ Listar estadios desde la base de datos
✔️ Eliminar estadios
✔️ Guardarlos de forma persistente con Room
✔️ Separación por capas (Clean Architecture)
✔️ Inyección de dependencias con Hilt



CONCLUSION

La implementación del módulo de estadios se realizó siguiendo la misma arquitectura del módulo de jugadores, reutilizando el patrón CRUD con Room, Hilt y Clean Architecture.
Esto permitió:
Escalabilidad del sistema
Código organizado por capas
Separación de responsabilidades
Persistencia de datos local en SQLite mediante Room